



Pizza42 + Auth0

Modern CIAM Architecture & Pre-Sales Solution
Presentation

A technical walkthrough of how Pizza42 leverages Auth0 to deliver secure, frictionless customer identity — from Universal Login to scalable API authorization.

Business Goals

Pizza42's leadership identified four priorities. Auth0 directly addresses each.



Reduce Liability

Offload credential management entirely — no passwords stored, no breach exposure.



Frictionless Login

Enable social login, passkeys, and smooth onboarding experiences for every customer.



Modern Security

Improve security posture with OAuth 2.0 and OIDC industry standards.



Customer Enrichment

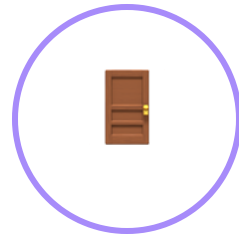
Progressive profiling and social identity data enrich customer profiles to power personalized marketing.

High-Level Architecture

One identity platform. Five purposeful components. Clear trust boundaries.



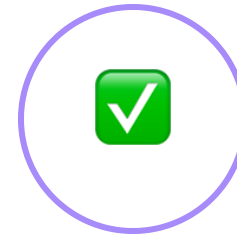
React SPA



Auth0 Login



Issue Tokens



Validate JWTs



Mgmt API (M2M)



React SPA

Universal Login integration for secure, centralized auth.



Auth0 API

Protects backend resources via JWT access tokens.



M2M Application

Isolated credentials for Management API access.



Post-Login Actions

Custom claims injected at runtime for richer token payloads.

Why Universal Login?

The single biggest security and time-to-market lever in this architecture.



Zero Password Handling

Pizza42 never touches customer credentials — Auth0 owns the entire authentication surface, removing PCI / breach exposure.



Built-In Security Features

MFA, password reset, social login, and passkeys are available out of the box with no custom code.



Faster Time to Market

Centralized auth reduces phishing risk and accelerates rollout — fully aligned with Auth0 best practices.

Pizza42 customers see one consistent, branded login page — phishing-resistant and ready for passkeys today.

OAuth 2.0 & OIDC Strategy

Every flow choice is intentional. Aligned with current IETF OAuth 2.0 Security BCP.

Authorization Code + PKCE SPA

Used by the React SPA. PKCE prevents authorization-code interception in browsers — the modern standard for public clients.

OIDC for Identity User claims

ID token carries who the user is — name, email, verified status — for personalized UI.

OAuth 2.0 for API Authz

Access token carries what the user is allowed to do. Backend validates and enforces scopes.

Client Credentials M2M

Server-to-server credentials never leave the backend. Used only for privileged Management API operations.

 Implicit Flow and ROPC explicitly rejected — known vulnerabilities, deprecated by IETF.

End-to-End Token Flow

Every step enforces a trust boundary. Privileged operations stay server-side.

1

User Authenticates

React SPA redirects to Universal Login.
User chooses email/password, Google, or
Passkey. Auth0 handles credentials.

2

Auth0 Issues Tokens

ID token to the SPA (identity), Access
token to the backend (authorization). Both
signed with RS256.

3

Backend Validates & Updates

API verifies JWT and scope. M2M token
used to write order to Auth0 user profile via
Management API.

The SPA never directly accesses the Management API — all privileged operations flow through the validated backend, ensuring credentials and sensitive operations stay server-side.

Security Controls

Defense in depth — multiple independent gates, each with one job.



JWT Validation

RS256 asymmetric signing verified via JWKS endpoint — no shared secrets.



Scope Enforcement

Backend explicitly checks `create:orders` on every request — authentication + authorization, not just one.



Email Verification

Server-side gate via Management API — always current, prevents unverified orders even if UI is bypassed.



Secrets Isolation

M2M client secrets never exposed to the browser — stored only in server environment variables.



Token Rotation

Refresh-token rotation with reuse detection — leaked tokens invalidate the entire family.



Tamper-proof Data

Orders stored in `app_metadata` — no user-self-service scope exists, structurally unwritable from the browser.

Scopes vs. RBAC

Start lean, scale to roles when the business needs it — no refactoring required.



Today: Lightweight Scopes

PoC uses fine-grained scopes like `create:orders` — simple, explicit, enforced in backend middleware.

Ideal for single-persona applications where role complexity is low.



Next: RBAC at Scale

As Pizza42 grows to support customer, staff, and admin personas, Auth0 RBAC turns on with a tenant-level toggle.

No frontend refactoring required — backend already checks both scope and permissions claims.

✅ Auth0 RBAC can be enabled at the tenant level with no frontend refactoring required.

Business Value Mapping

One identity platform — value across every team Pizza42 cares about.



Security

Eliminates credential liability and reduces attack surface — zero password storage.



Product

Frictionless login, social sign-in, passkeys — drives conversion and retention.



Marketing

Progressive profiling enriches customer data over time — no friction at signup.



Operations

Fewer password reset tickets and self-service recovery reduce support burden.



Future AI

Rich identity data powers personalized reorder recommendations and GenAI experiences.

Every stakeholder benefits from the same investment. That's why Auth0 is a platform decision, not a feature decision.

Production Roadmap

A phased plan from PoC to enterprise-scale deployment.

1

Phase 1 — Data Foundation

Migrate orders to a dedicated production database. `user_metadata` mirrors a thin summary for token use.

2

Phase 2 — Brand & Token Hardening

Custom Auth0 domain (`auth.pizza42.com`) for brand consistency. Refresh-token rotation with reuse detection on.

3

Phase 3 — Adaptive Risk

Adaptive MFA + breached-password detection + anomaly detection — respond to real-time risk signals.

4

Phase 4 — Observability & Scale

Auth0 log streams to SIEM. Progressive profiling Action drives marketing enrichment. Passkeys default for new signups.

Demo Walkthrough

What you'll see live, in roughly 20 minutes.

01

Signup & Email Verification

New user registration via Universal Login. Server-side verification gate blocks orders until email is confirmed.

02

Order Placement Flow

Authenticated SPA submits an order with the access token. Backend validates JWT, scope, and email status before persisting.

03

JWT Inspection

Decode and inspect token claims, audience, issuer, and enforced scopes live. Custom claim shows recent orders & marketing profile.

04

Social Login & Dashboard Tour

One-click Google login skips the verification step entirely. Auth0 dashboard tour shows API record, M2M app, and Post-Login Action.

Key Takeaways

What Pizza42 walks away with.



Best-Practice Architecture

Built on Auth0's recommended patterns — PKCE, Universal Login, M2M isolation, JWT validation. Actions (not deprecated Rules).



UX + Security in Balance

Passkeys and social login deliver a smooth experience without compromising security posture.



Enterprise-Scale Foundation

Scopes-to-RBAC upgrade path, log streaming, adaptive MFA — ready to activate at scale.



Built for All Stakeholders

Technical depth for architects. Clear business value mapping for product, marketing, and executive teams.

Architecture Flow Diagram

Every trust boundary is enforced. Privileged operations stay server-side.

